固定されたゲージを、

腕のカメラで撮影・点検する

ゲージが画像の端にずれて写ると、そのままでは針の状態を判定しにくくなります。この作品は、**画像を見て腕を少し回し、点検に適した画像を選んでから、正常・異常に分類する**仕組みです。対象は、針の向きで値を示す計器（ゲージ）を模した、仮想空間の固定物体 1 台です。

2 つの学習モデルの役割

学習とは、用意した例から、入力と正解の対応を表す数値を求めることです。モデルとは、その数値を使って新しい入力への答えを計算する仕組みです。

撮影位置を合わせるモデル	画像の左右ずれと現在の関節角から、次に腕をどちらへどれだけ回すかを予測します。**模倣学習**で、正解として与えた操作例に近い補正量を学びます。
画像を分類するモデル	撮影画像から輪郭の数値を作り、**線形 SVM** で正常・異常を選びます。針位置のラベル付き画像で学習済みです。

人が用意し、人が開始する

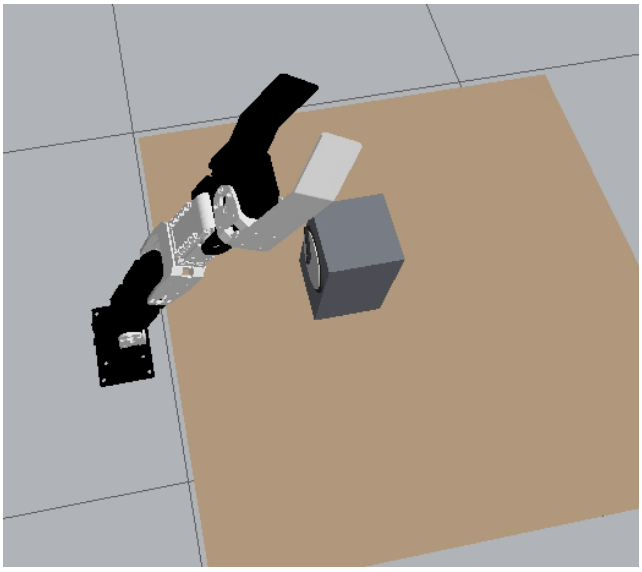
人がシミュレータと、プログラム間で情報・命令を受け渡す ROS などの制御環境を起動し、ゲージの配置・針角、使用モデル、撮影条件、安全設定を準備します。その後、**Python の実行コマンドを人が開始**します。1 回の点検試行を、記録では run（ラン）と呼びます。

開始後の補正方向・補正量を人が 1 回ずつ選ぶ方式ではありません。プログラムが観測、補正の判断、安全確認、動作、撮影画像の採用、分類、所定位置への復帰を順に進めます。

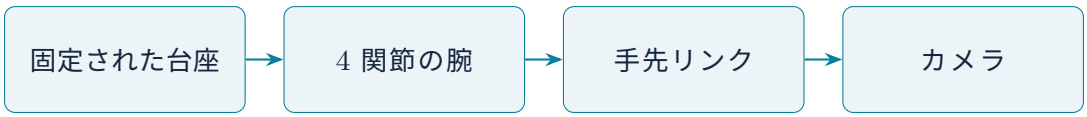
次へ渡すもの　次ページで装置の位置関係を確認し、その後、1 回の点検がどの順で進むかを追います。数式は、各処理が何を求めるかを説明してから示します。

箱を固定し、腕と手先カメラを動かす

なぜ腕を動かすのか。目的は、ゲージそのものを移すことではなく、**判定しやすい写り方へカメラの視点を変える**ことです。Gazebo Fortress は、ロボット・物体・カメラを仮想空間内で動かし、物理状態や画像を計算するシミュレータです。



採用した異常例と同じ試行の保存済み外部録画から、装置部分を切り出した静止画。装置の説明用であり、後の手先画像と同時刻の比較図ではない。



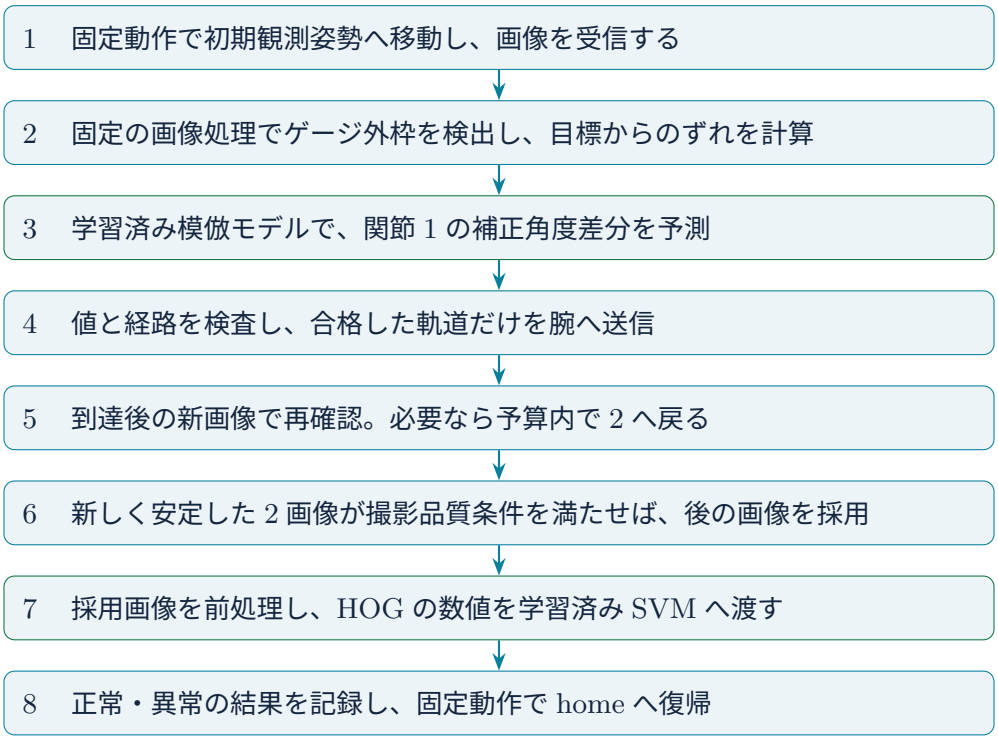
使用した腕は CRANE+ という 4 関節アームです。固定 joint（ジョイント）は、手先とカメラの取付位置・向きを変えない接続です。角度の単位 rad（ラジアン）は π rad が 180 度です。カメラは手先リンクに対して x 方向 32.5 mm、y 軸回り 0.30 rad の取付関係です。関節 1 が回ると、つながった手先とカメラの姿勢も変わります。

画像を構成する小さな点を画素と呼び、数を px で表します。カメラは 640×480 画素、水平画角 1.1 rad、設定上 10 Hz（1 秒 10 回）です。箱の位置・姿勢は点検開始前に決め、点検中は世界座標で固定します。世界座標は仮想空間内の位置を表す m 単位の値です。画像内の px 単位の位置とは別です。

次へ渡すもの　同じ手先カメラから受け取る画像と、腕の関節角の実測値を、位置補正の判断へ渡します。

開始後、1 回の点検はこうつながる

画像にゲージが写っていることと、針位置が正常であることは別です。まずゲージの**場所を検出**して撮影位置を整え、次に採用画像の**状態を分類**します。



緑の枠は学習済みモデルを使う処理です。その他の画像処理、撮影条件、安全確認、初期移動・復帰は、決めておいた処理です。**学習は点検前に済ませ、点検中にモデルの係数を変更しません。**

次へ渡すもの　次ページで開始・終了の条件を確認します。その後は 2 の外枠検出から、各箱の計算を順番に説明します。

何を確認して開始し、どの条件で終えるか

動作の計算が正しくても、通信先が違ったり、古い関節状態を読んだりすれば、そのまま腕を動かせません。人の開始操作を受けた実行プログラムは、動作前の状態確認を通してから処理を進めます。

開始時の確認と、自動化の範囲

専用の実行プログラムが準備・記録を開始し、その中から動作用プログラムを起動します。通信環境、使用する動作実装、排他ロック、新鮮な関節状態、controller、命令の受信先、必要サービス、対象状態、home にいることなどを確認します。

排他ロックは、同時に別の実行者が命令を送らないための仕組みです。controller は、指定された関節角と時刻の列に沿って腕を動かす制御系です。home は、あらかじめ決めた終了・開始用の関節姿勢です。

正常終了と停止

撮影品質を満たす画像を採用し、分類結果を記録して home への到達を確認したら、点検完了です。撮影品質が不足した場合は未分類のまま記録し、状態が明確で復帰条件を満たすときに home へ戻ります。

通信・controller・経路計画・衝突検査・到達確認の異常では、その後の命令送信を止めます。状態不明のまま home を追加送信しません。要求・命令の自動再送で成功扱いにする処理もありません。

補正 0 回の正常例と矛盾しない条件説明

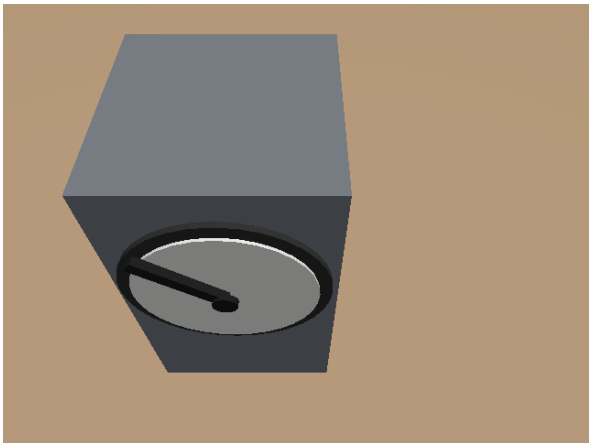
本資料の**水平補正の実験用実行プログラム**は、初回画像で水平ずれが 35 px 以内なら「補正が必要な条件の試行ではない」として分類せず終了・復帰します。性能実験の設計条件を守るための分岐です。

動画の正常例は、**初回から撮影品質を満たせば分類へ進める別の実行経路**の保存記録です。安定した画像を採用したため、模倣モデルの呼出しも補正も 0 回でした。すべての記録に一つの分岐規則が適用されていたとは扱いません。

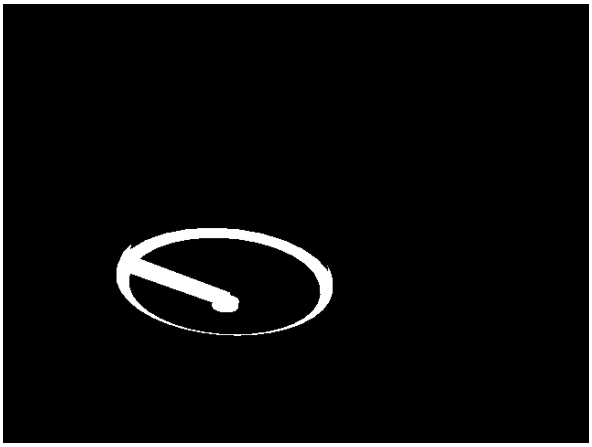
> **次へ渡すもの**　動作可能な条件を満たした後、受信した 640×480 画像を検出処理へ渡します。

まず、画像からゲージの候補を探す

解決したい問題は、画像のどこを基準に補正すればよいかを求めることです。前の処理から受け取るのは、同じ手先カメラの 640×480・BGR8 画像です。BGR8 は青・緑・赤の明るさを、それぞれ 0〜255 の整数で表す形式です。



保存済みの元画像



暗い画素だけを白で表示

右は左の保存画像を録画後に説明用に処理したもの。新しい撮影や分類結果ではない。

暗い画素を、つながったまとまりへ変える

まず色画像をグレースケール、つまり各画素を 1 個の明るさで表す画像へ変換します。明るさを $G(u, v)$ とすると、暗い画素の印 $M(u, v)$ は次です。

$$M(u,v) = \begin{cases} 1 & G(u,v) < 40 \\ 0 & G(u,v) \geq 40 \end{cases}$$

u, v は画像内の横・縦位置です。式は「明るさ 40 未満なら暗い画素として残す」と読みます。例えば明るさ 25 は 1、80 は 0 になります。

1 の画素のうち、上下左右または斜め隣でつながるものを、1 つの**連結成分**と呼びます。プログラムは暗い画素のまとまりを列挙し、それぞれの全体を囲む候補長方形を作ります。候補の位置と大きさは画像ごとに変わります。

次へ渡すもの　候補ごとの左上位置、幅、高さ、暗いまとまりの画素数を、次ページの条件検査へ渡します。この時点では、まだ針の正常・異常を判定していません。

候補を条件で除き、1 個だけ残れば採用する

暗い部品や影も候補になり得ます。そのため、前ページで得た**すべての候補長方形**を、ゲージ外枠らしい形・大きさ・位置かという固定条件で検査します。固定なのは合格範囲であり、検出する長方形自体ではありません。

調べる値	合格条件と意味
幅 w、高さ h	$70 \le w \le 300$ px、$h \ge 25$ px。極端な大小のまとまりを除く。
縦横比 w/h	$1.45 \le w/h \le 2.6$。外枠の見かけの形を検査する。
暗画素割合 ρ	$0.12 \le \rho \le 0.65$。下の式でまとまりの密度を見る。
画像端	$x \ge 8,\ y \ge 8,\ x+w \le 632,\ y+h \le 472$。端の断片を避ける。
残る候補数	ちょうど 1 個なら採用。0 個または複数なら不採用。

$$\rho = \frac{A_{\text{component}}}{wh} = \frac{6813}{236 \times 118} \simeq 0.245$$

$A_{\text{component}}$ は、**その 1 つの連結成分に属する暗画素数**です。分母 wh は候補長方形全体の画素数です。長方形内にある別の暗い成分まで数え直す方式ではありません。例は約 24.5% を占めるという意味で、確率や AI の信頼度ではありません。

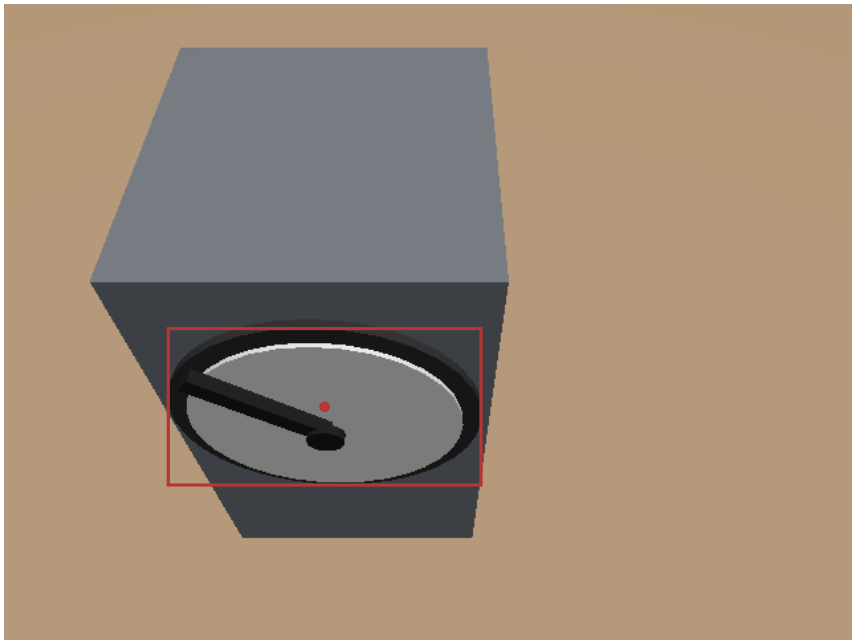
左上を (x, y)、幅を w とすると、含まれる横の画素番号は x から $x+w-1$ までです。条件の $x+w$ は、右端の画素の**次の座標**を使っています。縦も同様です。

条件後も複数残った場合、順位付けして一番を選ぶ処理はありません。どれを動作基準にすべきか一意に決められないため、検出不成立として次の補正へ進めません。

次へ渡すもの　候補が 1 個なら、その長方形と中心を画像座標で出力します。次ページで実記録の座標を確かめます。

実記録の長方形から、中心を計算する

補正したいのは、ゲージ外枠の中心が撮影目標からどれだけずれているかです。ここでは保存された検出記録 $(x, y, w, h) = (124, 245, 236, 118)$ を使います。PDF の見た目から枠を推定した値ではありません。



赤枠・赤点は保存検出情報を重ねた編集注釈。元画像は保持。

含まれる画素番号の四隅は、左上 $(124, 245)$、右上 $(359, 245)$、右下 $(359, 362)$、左下 $(124, 362)$ です。右端・下端の次の座標は $(360, 363)$ になります。

$$u_{\text{gauge}} = x + \frac{w}{2} = 124 + \frac{236}{2} = 242\,\text{px}$$

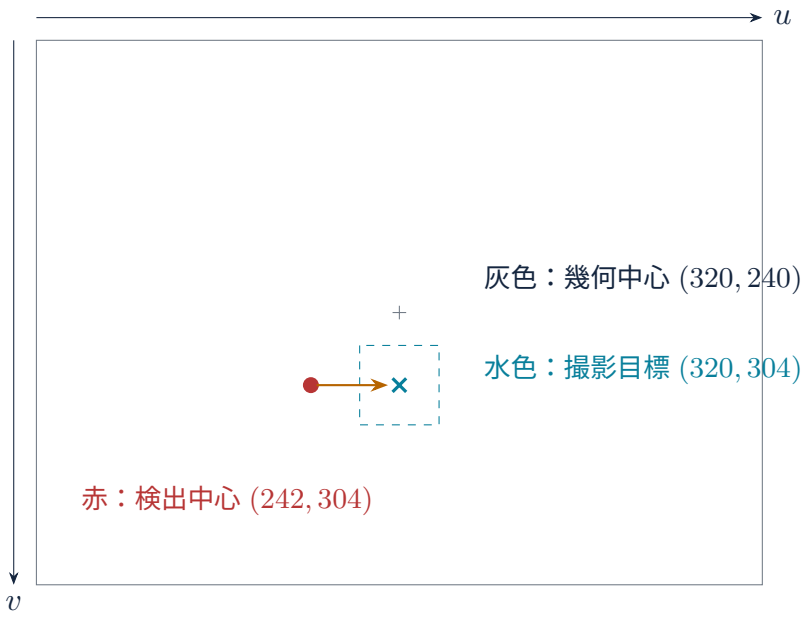
$$v_{\text{gauge}} = y + \frac{h}{2} = 245 + \frac{118}{2} = 304\,\text{px}$$

式は「長方形の左上から、幅と高さの半分だけ進んだ位置を中心にする」と読みます。コードは画素数 w, h の半分を使い、例えば高さが奇数なら中心は 0.5 px 刻みになり得ます。

次へ渡すもの　中心 $(242, 304)$ px と外枠幅 236 px を出力します。次に、同じ画像内に設定した撮影目標との差を計算します。

画像のどの点へ、近づけたいのか

画像の左上を原点 $(0, 0)$、右を u の正方向、下を v の正方向とします。下図の点はすべて同じ 640×480 画像内の座標です。



座標を説明する図。矢印は画像内で近づけたい方向であり、カメラの実空間での移動方向ではない。

灰色と水色は画像上の固定点です。箱を世界に固定したまま腕・カメラを動かすと、**画像内の赤い中心**が変わります。赤を水色の許容範囲へ近づけます。完全一致や、灰色との一致は必要ありません。

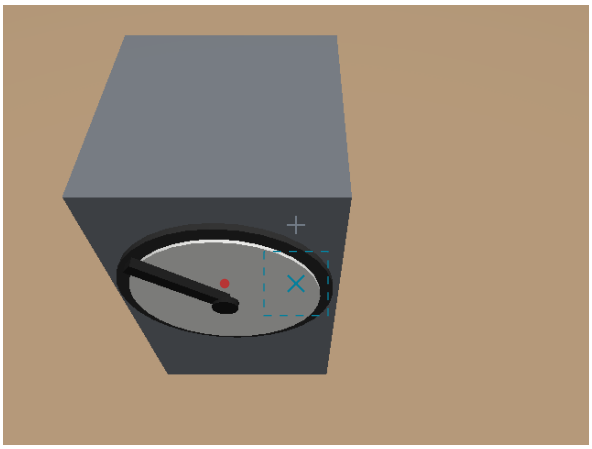
$$e_x = u_{\text{gauge}} - 320 = 242 - 320 = -78\,\text{px}$$

e_x は左右ずれで、負なら目標より左、正なら右です。水色の目標は幾何中心より下へ 64 px です。この設定値は確認できましたが、選定理由を裏付ける設計記録はなく、理由は断定しません。

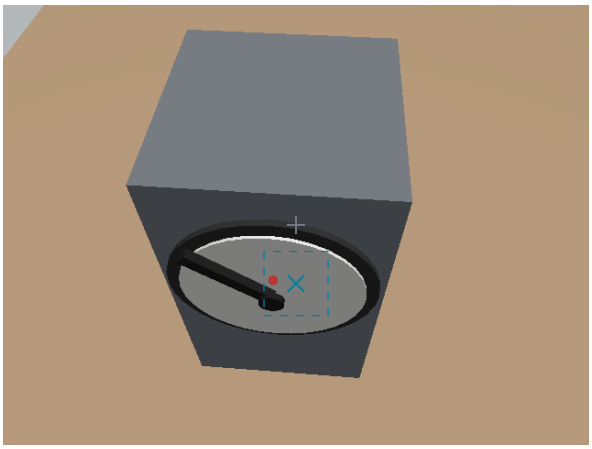
> **次へ渡すもの**　左右ずれ e_x [px] を、現在の関節 1 角 q_1 [rad] と組にします。px、世界の m、関節角の rad を同じ単位として扱いません。

同じ箱を撮っても、視点が変われば画像内の位置が変わる

画像座標と実空間の動きの違いを、同じ保存試行の補正前後で確かめます。画像の寸法はどちらも 640×480 で、同一縮尺で表示しています。箱は試行中固定し、腕と取り付けたカメラの向きが変わりました。



補正前：$e_x = -78$ px



2 回補正後：$e_x = -25$ px

赤点：保存検出中心。水色の $\times$：撮影目標。水色の破線：中心の許容範囲。灰色の＋：画像幾何中心。すべて録画後の編集注釈。

赤い中心は $(242, 304)$ から $(295, 300.5)$ px へ変わりました。横方向では右へ 53 px 近づき、目標からの左右ずれの絶対値は 78 から 25 px へ減りました。水色と灰色の固定点へ箱を物理的に移したという意味ではありません。

この間、1 回目の再観測では左右ずれが -46 px で、まだ許容値 35 px を超えていました。次の画像からもう一度補正を選び、2 回目の後は -25 px になりました。同じ命令を再送したのではなく、各回の新しい画像から次の操作を選んだ記録です。

ここで図示した水色の枠は中心ずれの条件だけです。対象の大きさ、画像の新しさ、2 枚の安定性も必要なので、赤点が枠内に入ったことだけで撮影品質全体の合格は決まりません。

次へ渡すもの　ここまでで、補正のための観測値が何かを確認しました。次は、この状態に対する学習用の正解操作をどう作るかを説明します。

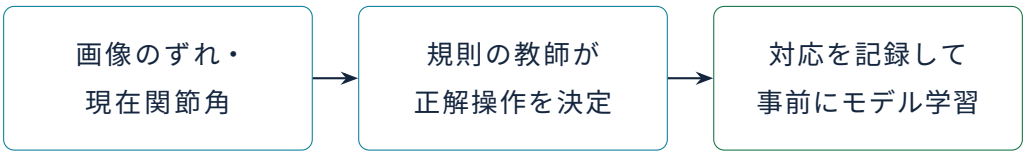
学習用の正解操作を、計算ルールで作る

モデルが腕の補正を学ぶには、「この写り方と関節角なら、どの操作を選ぶか」という例が必要です。**教師**とは、学習時に正解操作を与える役割です。今回は、人の手動操作ではなく、人が決めた計算ルールに従う**規則制御**を教師にしました。

1 つの教師操作を記録するまで

1. ゲージを学習用配置へ置き、点検開始前に固定する。
2. 手先画像から外枠を検出し、水平ずれ e_x [px] を測る。
3. 関節センサから、操作直前の関節 1 角 q_1 [rad] を読む。
4. 教師の計算式で、関節 1 を回す差分角度を決める。
5. 出力と経路の安全検査に合格した場合にだけ動かす。
6. 到達した関節角と、その後の新しい画像を保存する。
7. 到達・次画像などの条件が確認できた操作だけを、状態と教師補正量の組として学習に使う。

このように、観測状態と教師操作の組から、同じような操作を出す**方策**を教師あり学習する方法が模倣学習です。方策とは「今見えている状態から、次の操作を選ぶ決め方」です。今回は behavior cloning と呼ばれる方法です。[1]



次へ渡すもの　次ページで教師の補正量を求めます。学習後の点検時には、教師式の代わりに保存済み回帰モデルを呼び、規則教師へ自動で切り替えません。

画像のずれを、腕の角度差分へ換算する

教師が求めたいのは、「画像上のずれを小さくするために、関節 1 を今から何 rad 回すか」です。画像のずれ e_x [px] を、そのまま関節角にはできないため、画像位置と角度の換算係数 J を使います。

換算係数は、保存された小さな試し動作から求めた

初期観測姿勢を基準に関節 1 へ $+0.035$ rad を要求した測定では、外枠中心の横座標が 320 から 344 px へ変わりました。

$$J = \frac{344 - 320}{0.035} = 685.7142857 \text{ px/rad}$$

これは「この観測姿勢付近で、関節 1 の角度を 1 rad 変えたときの画像位置の変化量へ換算した値」です。**要求した試し角度を分母にする有限差分**で、広い配置・角度で一定と検証した値ではありません。

この測定から何が読み取れるか

関節 1 の正方向の動作に対し、画像の対象中心も右へ変わり、J は正でした。そこで、画像中心が目標より左なら、正方向の補正が候補になります。ただし、測定した付近の小さな変化の関係を使っているため、次の新画像で実際のずれを確認する必要があります。

単位の換算も確かめます。$42\,\text{px}/(685.7142857\ \text{px/rad}) = 0.06125\,\text{rad}$ です。px が分子・分母で打ち消され、腕を動かす角度の単位が残ります。これは「42 画素を消すための局所的な角度の見積もり」で、到達成功の保証ではありません。

次へ渡すもの　画像変化と角度の換算値 J を、教師操作の比例式へ渡します。次ページでゲインと符号を含めて、実記録の -42 px の例を計算します。

教師は、ずれを減らす向きと量を計算する

前ページの換算値 J と、今回の画像で測った左右ずれ e_x を受け取ります。求めたいのは、関節 1 を今から回す**角度差分**です。初期観測姿勢の絶対角そのものを予測する式ではありません。

教師の式を一つずつ読む

$$e_x = u_{\text{gauge}} - 320, \qquad a^{\mathrm{T}}_{\text{raw}} = -k\frac{e_x}{J}, \quad k = 0.5$$

上付き T は教師を表す記号です。行列の転置を表す $\top$ とは区別します。e_x/J の単位は $\text{px}/(\text{px}/\text{rad}) = \text{rad}$ となるので、画像のずれを角度へ換算できます。マイナス符号は、この正方向の対応でずれを減らす向きに動くためです。$k = 0.5$ は、換算した全量を一度に補正しない設定です。

$$e_x = -42\,\text{px} \quad \Rightarrow \quad a^{\mathrm{T}}_{\text{raw}} = -0.5\frac{-42}{685.7142857} = +0.030625\,\text{rad}$$

非有限値や $|a^{\mathrm{T}}_{\text{raw}}| > 0.11$ rad は拒否します。それ以外は安全上限 ± 0.055 rad と、要求量の上限 ± 0.0548 rad を順に適用します。通過した値の最終的な教師ラベルは

$$a^{\mathrm{T}} = \text{clip}(a^{\mathrm{T}}_{\text{raw}}, -0.0548, +0.0548)$$

です。clip は下限未満を下限、上限超過を上限へ置き換える操作です。この例では値は変わりません。

> **次へ渡すもの**　制限後の要求差分 $+0.030625$ rad を教師ラベルとして記録します。実際に到達した角度との差分を教師ラベルに置き換えてはいません。

11 回の点検から、17 件の操作例を得た

1 回の点検試行は、初期移動、観測、必要な補正、撮影画像の採用、分類、home 復帰までを含みます。これを学習の記録では 1**episode（エピソード）**と数えます。一方、**1 操作レコード**は、その中の補正 1 回について、状態・教師補正量・到達・次画像を対応させた記録です。

左右それぞれ小・中・大の 6 配置で 12 試行を予定し、11 試行が完了しました。1 試行の故障停止は残し、学習例に含めていません。11 完了試行の中に、1 回補正と複数回補正があったため、有効な教師操作は 17 件です。

実記録の例	e_x [px]	q_1 [rad]	教師差分 [rad]
左・小の 1 操作	-42	$+0.00007862$	$+0.030625$
右・小の 1 操作	$+42$	-0.00006120	-0.030625
左・中の 1 操作	-64	-0.00008920	$+0.046667$
右・大の 1 操作	$+114.5$	-0.00009009	-0.054800

保存済み教師レコードの抜粋。丸めて表示。説明用に作った架空のデータではない。

学習例全体を D、操作の番号を $i = 1, \dots, 17$ とし、次のように表します。

$$D = \{(\boldsymbol{x}_i, a_i^{\mathrm{T}})\}_{i=1}^{17}, \qquad \boldsymbol{x}_i = \begin{bmatrix} e_{x,i} \\ q_{1,i} \end{bmatrix} \in \mathbb{R}^2$$

$\boldsymbol{x}_i$ は 2 個の入力値を縦に並べた列ベクトルです。$[e_{x,i}, q_{1,i}]^{\top}$ とも書け、$\top$ は横と縦を入れ替える転置です。a_i^{T} は関節 1 の教師補正量で、1 個の実数です。

教師の比例式自体は q_1 を直接使いません。学習モデルは e_x と q_1 の両方を受け取る実装です。現在姿勢を入力できる設計ですが、**q_1 を加えることの有効性を独立した比較で検証した記録はありません。**

次へ渡すもの　17 組の 2 次元入力と教師差分を、学習計算へ渡します。入力に箱の真の世界位置や正常・異常ラベルは入れません。

入力数値の尺度をそろえてから学習する

左右ずれは数十〜百 px、関節角は 0.01 rad 程度です。このまま同じ式に入れると、係数への罰則が入力の単位の大きさに強く左右されます。そこで 2 個の**入力数値**を、学習例の平均からどれだけ離れた値かへ換算します。これが**標準化**です。元画像を加工する処理ではありません。

入力の番号を j とし、$j=1$ は e_x [px]、$j=2$ は q_1 [rad] です。操作例 i の値を x_{ij} とします。

$$\mu_j = \frac{1}{17}\sum_{i=1}^{17} x_{ij}, \qquad s_j = \sqrt{\frac{1}{17}\sum_{i=1}^{17}(x_{ij}-\mu_j)^2}$$

総和記号 $\sum$ は 17 件分を足す意味です。平均 μ_j は値の中心、標準偏差 s_j は平均からの散らばりの大きさです。平均との差を二乗して平均し、その平方根を取るので、s_j の単位は元の値と同じです。実装は分母 17 の母標準偏差（NumPy の ddof=0）です。実記録の水平ずれ 17 件の和は 228.5 px です。その平均は $228.5/17 = 13.44117647$ px です。平均との差の二乗和は約 81360.4412 px^2 なので、標準偏差は $\sqrt{81360.4412/17} = 69.1802595$ px です。

$$z_{ij} = \frac{x_{ij}-\mu_j}{s_j}$$

式は「平均との差を、その入力の散らばりの大きさで割る」と読みます。z_{ij} は単位のない値になり、1 なら平均から標準偏差 1 個分だけ上です。

入力	平均 μ_j	標準偏差 s_j
左右ずれ [px]	13.44117647	69.18025950
関節 1 角 [rad]	−0.009716852	0.050571588

例えば保存入力 $e_x = -78$ px は、$(-78 - 13.44117647)/69.18025950 \simeq -1.32178$ へ変わります。もし $s_j < 10^{-6}$ なら分母を 1 へ置き換え、0 除算を避けます。今回の 2 入力では置換は起きていません。

次へ渡すもの　学習用 17 件だけから求めた μ, s を保存し、標準化した 2 値 z_{i1}, z_{i2} を予測式へ渡します。調整・評価時には平均と標準偏差を求め直しません。

教師の操作例に近い予測式を学ぶ

教師の式を毎回計算する代わりに、教師が実際に選んだ**状態と操作の対応をデータから近似する**ことが、この模倣学習の目的です。**回帰**は、入力から連続的な数値を予測する方法です。ここでは左右ずれと現在角から、教師の補正角度差分を予測します。

前ページの標準化した入力 z_{i1}, z_{i2} を受け取り、次の予測式を使います。

$$\hat{a}_i = b_{\mathrm{BC}} + w_{\mathrm{BC},1} z_{i1} + w_{\mathrm{BC},2} z_{i2}$$

$\hat{a}_i$ はモデルの予測差分 [rad] です。教師の正解差分 a_i^{T} とは別です。BC は behavior cloning を表す添字で、後に出る SVM の係数と区別します。

求めたい係数	予測への寄与
b_{BC}：切片 [rad]	2 入力が平均値、つまり $z_{i1} = z_{i2} = 0$ のときの予測。入力に掛けずに足す基準値。
$w_{\mathrm{BC},1}$ [rad]	標準化した左右ずれが 1 増えたとき、予測をいくら変えるか。
$w_{\mathrm{BC},2}$ [rad]	標準化した現在角が 1 増えたとき、予測をいくら変えるか。

入力が無次元なので、3 個の係数は予測差分と同じ rad 単位です。予測は「左右ずれの寄与」「現在角の寄与」「基準値」を足して作ります。

学習で決めたい未知数は、この 3 係数です。教師ラベルと予測の違いが小さくなるよう、17 例全体を使って値を決めます。学習済みの値は、およそ次です。

$$b_{\mathrm{BC}} = -0.00807696, \quad w_{\mathrm{BC},1} = -0.04071090, \quad w_{\mathrm{BC},2} = +0.00398139 \quad [\mathrm{rad}]$$

次へ渡すもの　次ページでは、何を「良い係数」とするかを二乗誤差と正則化で定めます。規則方式より便利・高性能になったと、この式だけで結論づけるものではありません。

教師と予測の違いを、17 件まとめて小さくする

1 例に合うだけでは、ほかの状態で教師と違う操作が出るかもしれません。そのため、前ページの予測式を 17 件すべてへ適用し、教師操作との差を合計します。最小化したい値を**損失**と呼びます。

$$E_i = (a_i^{\mathrm{T}} - \hat{a}_i)^2$$

a_i^{T} は教師差分、$\hat{a}_i$ は予測差分 [rad] です。例えば教師が $0.030625\ \mathrm{rad}$、予測が $0.028625\ \mathrm{rad}$ なら、差は $0.002\ \mathrm{rad}$、二乗誤差は $0.000004\ \mathrm{rad}^2$ です。逆向きの差でも二乗すれば正になり、大きな差ほど強く数えます。この数値例は**式の説明用**で、実測の予測誤差ではありません。

係数が過大になりにくいよう、追加の罰則を入れる

$$L = \sum_{i=1}^{17} (a_i^{\mathrm{T}} - \hat{a}_i)^2 + \lambda (w_{\mathrm{BC},1}^2 + w_{\mathrm{BC},2}^2)$$

第 1 項は 17 件分の二乗誤差の**総和**で、平均ではありません。第 2 項は大きな係数を不利にする**正則化**です。17 件への当てはまりだけで、極端な係数を選びにくくする役割があります。両項は rad^2、λ は単位のない設定値です。

λ は係数そのものではなく、罰則の強さです。例えば係数 $0.04\ \mathrm{rad}$ の二乗へ $\lambda = 0.01$ を掛けると $0.000016\ \mathrm{rad}^2$ の追加項になります。切片 b_{BC} には罰則を掛けません。平均付近の基準値まで 0 に寄せないためです。

二乗誤差と係数の二乗を組み合わせる回帰が**リッジ回帰**です。小さなデータでも学習計算を整えやすくしますが、撮影成功や未知配置への性能を保証するものではありません。

次へ渡すもの　この損失 L が最小になる 3 係数を求めます。次ページで、式を連立方程式に変えるところまで示します。

最小化を、3 係数の連立方程式に変える

17 本の予測をまとめて扱うため、数値を縦横に並べた**行列**を使います。$\boldsymbol{a}$ は教師差分を縦に並べた 17×1 列、$\boldsymbol{\beta}$ は求めたい係数の 3×1 列です。

$$\boldsymbol{\beta} = \begin{bmatrix} b_{\mathrm{BC}} \\ w_{\mathrm{BC},1} \\ w_{\mathrm{BC},2} \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & z_{11} & z_{12} \\ 1 & z_{21} & z_{22} \\ \vdots & \vdots & \vdots \\ 1 & z_{17,1} & z_{17,2} \end{bmatrix} \in \mathbb{R}^{17\times3}$$

1 行目と係数を掛けて足すと $1\times b_{\mathrm{BC}} + z_{11}w_{\mathrm{BC},1} + z_{12}w_{\mathrm{BC},2}$ になります。先頭の 1 は切片を足すためです。**行列積**はこの「行と列の同じ位置の積を足す」計算を全行へ行い、$Z\boldsymbol{\beta}$ は 17×1 の予測列になります。

$$L = (\boldsymbol{a} - Z\boldsymbol{\beta})^{\top}(\boldsymbol{a} - Z\boldsymbol{\beta}) + \boldsymbol{\beta}^{\top}R\boldsymbol{\beta}, \qquad R = \mathrm{diag}(0, \lambda, \lambda)$$

$\top$ は縦横を入れ替える転置、diag は指定した値を対角線に置き、ほかを 0 にする行列です。R は 3×3 で、先頭 0 によって切片を罰しません。転置した差の行と差の列の積が、17 件の二乗和です。

変化率が 0 になる条件を使う

最小点では各係数を少し変えたときの L の変化率が 0 になります。3 係数について微分すると、

$$\nabla_{\boldsymbol{\beta}} L = -2Z^{\top}(\boldsymbol{a} - Z\boldsymbol{\beta}) + 2R\boldsymbol{\beta} = \mathbf{0}$$

$\nabla_{\boldsymbol{\beta}} L$ は 3 係数それぞれの微分を縦に並べた列で、右辺は 3 個とも 0 という意味です。上の式を展開して 2 で割り、未知係数の項を左へ集めると、

$$(Z^{\top}Z + R)\boldsymbol{\beta} = Z^{\top}\boldsymbol{a}$$

です。左の行列は 3×3、右は 3×1 です。この 3 元連立一次方程式を NumPy の `linalg.solve` で解きます。逆行列を直接作る実装ではありません。

次へ渡すもの　計算された 3 係数と入力の平均・標準偏差をモデルとして保存します。次ページでは罰則の強さ λ を何で選んだか説明します。

調整用の別データで、罰則の強さを選ぶ

学習 17 操作への当てはまりだけで設定を選ぶと、その例に都合のよい設定になり得ます。そこで、係数を求める学習配置とは別の左右 2 配置から得た 4 教師操作を、**調整**に使いました。調整とは、学習方法の設定を選ぶための確認です。

事前に定めた候補 $\lambda = 0.01, 0.1, 1.0$ について、学習 17 件だけで係数と標準化値を求め、調整 4 操作への予測を比べました。以下は保存された学習時の設定選択の結果であり、点検中に学習する処理ではありません。

$$\mathrm{MAE} = \frac{1}{4}\sum_{i=1}^{4} \left| a_i^{\mathrm{T}} - \mathrm{clip}(\hat{a}_i, -0.0548, +0.0548) \right|$$

MAE は平均絶対誤差です。ここでは「教師差分と、要求上限を適用した予測差分の差の絶対値を 4 件で平均する」と読み、単位は rad です。二乗で学習した損失と、設定選択に使ったこの指標は別です。

候補 λ	調整 4 操作の MAE [rad]
0.01（採用）	0.00226927
0.1	0.00248167
1.0	0.00448370

最小の 0.01 を採用しました。同じ誤差なら小さい λ を選ぶ規則です。最終比較の画像・操作・結果は、この選択や標準化に入れていません。

この実証が示すのは、教師の状態→操作対応を、データから求めた小さな予測式で近似できた範囲です。単純な教師式なら、その式を直接使う規則制御も有効な比較相手です。学習したという理由だけで優れているとはしません。

次へ渡すもの　選んだ係数・標準化・上限を固定し、点検時には新しい観測から予測値だけを計算します。この計算を推論と呼びます。

学習済み係数で、次に回す角度を求める

推論は、学習で決めた係数を使って新しい入力への答えを計算することです。ここで求めるのは、保存された異常例の最初の観測に対する、関節 1 の次の補正差分です。係数を更新する処理はありません。

1　観測した 2 値を受け取る

$$\boldsymbol{x} = \begin{bmatrix} -78\,\mathrm{px} \\ -0.00000897515\,\mathrm{rad} \end{bmatrix}$$

上は画像の左右ずれ、下は操作直前の実測関節 1 角です。

2　学習時に保存した平均と標準偏差で標準化

$$z_1 = \frac{-78 - 13.44117647}{69.18025950} \simeq -1.32178135$$

$$z_2 = \frac{-0.00000897515 - (-0.00971685179)}{0.05057158842} \simeq 0.19196306$$

どちらも単位のない値です。観測画像の画素を変えているわけではありません。

3　保存係数を掛けて足す

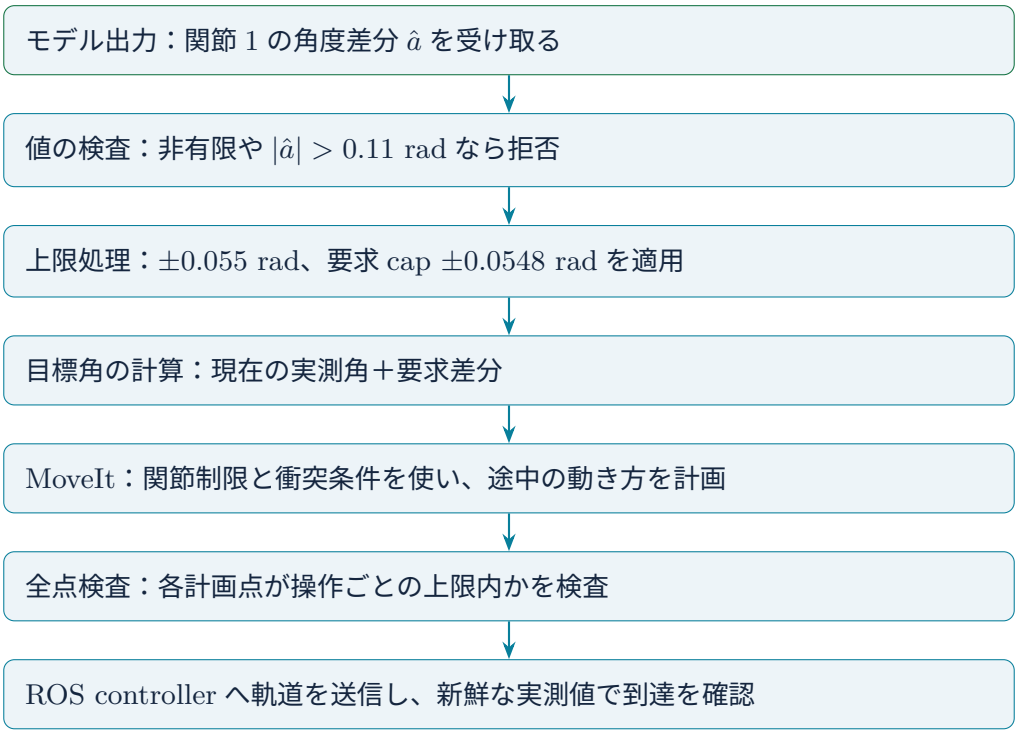
$$\begin{aligned} \hat{a} &= b_{\mathrm{BC}} + w_{\mathrm{BC},1} z_1 + w_{\mathrm{BC},2} z_2 \\ &\simeq -0.00807696 + (-0.04071090)(-1.32178135) \\ &\quad + (0.00398139)(0.19196306) \\ &= +0.04649823\,\mathrm{rad} \simeq +2.6642^\circ \end{aligned}$$

度への換算は $\hat{a} \times 180/\pi$ です。**これは今から正方向へ約 2.66 度回すという予測**であり、回転後の絶対角ではありません。保存入力と保存モデルを使った再計算であり、新しい観測・動作・学習ではない。元の実行記録の raw 出力 $+0.046498233$ rad とも一致した。

次へ渡すもの　予測差分を出力検査へ渡します。予測はそのままモータへの命令になるわけではありません。

予測を、安全確認を通る動作へ変える

モデルが返すのは角度 1 個です。途中でどのように腕を動かすか、物体へ接触しないかは、その数値だけでは決まりません。ここからは学習モデル以外の固定処理が担当します。



MoveIt は腕の動き方を計画するソフトウェアです。**軌道**とは、途中の関節角と時刻を順に並べたものです。controller は、その軌道に沿って関節を動かす制御系です。ROS は、これらのプログラム間で情報や命令を受け渡す仕組みです。

clip による数値の制限、危険な出力の拒否、計画した経路の検査は、それぞれ別の処理です。最終点が範囲内でも途中が範囲を超える場合があるため、**検査対象の全計画点**を調べます。

次へ渡すもの　検査に合格した軌道だけを送ります。不合格は診断情報を保存し、軌道を切り詰めたり無検査で送ったりせず停止します。

予測差分・目標角・送信終点・実測角は違う

前ページの各値を混同しないため、同じ保存操作で数値を追います。すべて関節 1 の値です。角度差分は「今から何 rad 回すか」、絶対角は「基準姿勢から見て何 rad の角にするか」です。

この操作の値	保存値 [rad]
操作直前の実測角 q_1	-0.00000897515
モデルの予測差分 $\hat{a}$	$+0.04649823305$
上限適用後の要求差分 Δq_1	$+0.04649823305$
要求した絶対目標 $q_1 + \Delta q_1$	$+0.04648925791$
計画して実際に送った軌道の終点	$+0.04657373408$
到達確認時の実測角	$+0.04656364708$

この例の予測は要求上限より小さいので、制限後も同じです。計画終点と要求目標の小さな違いは、計画の数値許容内の違いです。実測角は別途、関節状態から確認した値です。予測・要求・計画・到達を同じ数値として書き換えません。

途中の点を何と比較するか

$$|q_{1,k}^{\mathrm{plan}} - q_1^{\mathrm{start}}| \le 0.055 + 10^{-9}\ \mathrm{rad}$$

k は計画点番号です。q^{start} は**その操作の計画開始時の新鮮な実測状態**で、最初の home や初回観測姿勢ではありません。補正しない関節 2〜4 も、各点で開始状態との差を 0.002 rad 以内に制限します。

要求 cap の 0.0548 rad は、計画中の微小な数値変化に余裕を持たせるため、安全上限 0.055 rad より少し小さくした要求値です。全点検査の上限は増やしていません。送信直前の実測状態、関節の対応、衝突・到達の確認も別に行います。

次へ渡すもの　到達と静止が確認できた後、その到達より新しい手先画像を取得します。初期移動と home 復帰は固定動作で、この水平補正モデルが選ぶ操作ではありません。

受信した画像から、点検に使う画像を選ぶ

ここでいう点検は、**ゲージの針位置を正常・異常に分類すること**です。画像を受信しただけでは、分類に適した画像を確保できたとは限りません。補正判断に使う観測画像のうち、次の撮影品質確認を通ったものを**検査用画像として採用**します。

確認	実際の条件	避けたい問題
形式	640×480 BGR8、全画像の明るさの標準偏差 ≥ 4	空やほぼ単色の画像
検出	条件後の外枠候補が 1 個、枠全体が画像内	不明な対象や端の断片
中心	$(320, 304)$ から横・縦それぞれ 35 px 以内	分類しにくい照準ずれ
大きさ	検出枠幅 145〜255 px	小さすぎ・大きすぎる写り方
鮮度	到達後に受信し、画像の sim 時刻も到達の状態時刻より後	動く前の古い画像
安定性	品質合格の新しい 2 画像で、中心の各軸変化 ≤ 6 px、幅変化 ≤ 6 px	まだ落ち着かない観測

sim 時刻はシミュレータ内の時刻です。PC 上の受信時刻とは区別し、両方で鮮度を確認します。2 枚の撮影時刻と受信時刻が進んでいることも必要です。画像待ちは 15 秒を上限とし、取得コマンドの自動再送はしません。

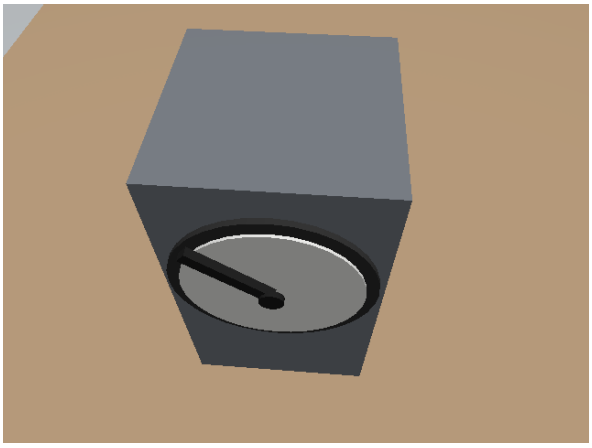
保存した異常例では、2 回補正後の中心は $(295, 300.5)$ px、目標との差は $(-25, -3.5)$ px、幅 234 px でした。続く画像でも同じ中心・幅で、撮影品質条件を満たしました。

撮影品質 OK は、ゲージ正常という意味ではありません。よく写った画像から異常と分類されることもあります。未撮影や分類未実施を正常として数えません。品質不足は結果として記録し、通信などの状態不明の故障と区別します。

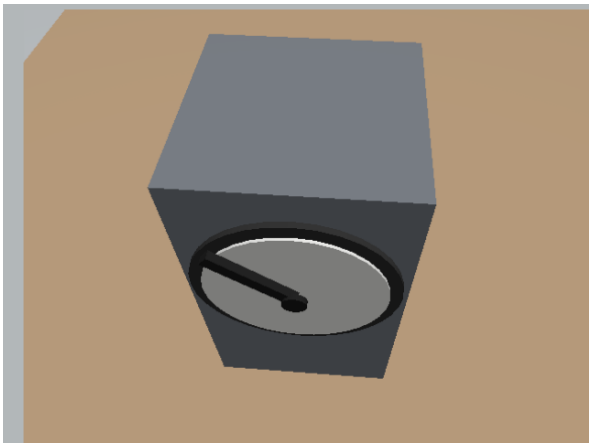
次へ渡すもの　2 枚の安定条件を満たした後の元画像と、その検出枠を分類前処理へ渡します。元画像は変更せず保存します。

分類前に、画像内の位置と大きさをそろえる

カメラが動くと、同じ針でも画像上の位置・見かけの大きさが変わります。分類器へ渡す前に、採用画像の検出枠を基準枠へ合わせ、分類に使う範囲を安定させます。これは**画像の加工**であり、箱やカメラを動かす操作ではありません。



採用した元画像



保存済みの分類用位置合わせ画像

元の検出枠の左上を (x, y)、幅と高さを w, h とし、元画像の点 (u, v) を加工画像の点 (u', v') へ移します。すべて px 単位です。

$$u' = 202 + \frac{236}{w}(u - x), \qquad v' = 245 + \frac{118}{h}(v - y)$$

式は**元枠の左上を引く→基準枠に合う倍率で拡縮→基準枠の左上へ移す**と読みます。横と縦の倍率は別で、回転や遠近のゆがみまで補正する式ではありません。

この採用画像の検出枠は $(178, 241, 234, 119)$ です。左上を代入すると $(u', v') = (202, 245)$、右下の**次の境界** $(412, 360)$ を代入すると $(438, 363)$ になります。幅 236、高さ 118 の基準枠へ対応します。

加工後も 640×480 です。線形補間は、拡縮後の画素の明るさを近くの画素から計算する処理で、撮影時刻の間の画像を作る時間補間とは違います。画像外は BGR 値 $(185, 183, 178)$ で埋めます。

次へ渡すもの　位置合わせ画像から、次ページの固定領域を切り出します。撮影の証拠としての元画像と、分類へ渡した加工画像は別々に追跡します。

分類に使う範囲を切り出し、128×128 にする

前ページの位置合わせ画像には周囲の箱や背景も含まれます。そこで、針と外枠の見た目を数値化するため、**分類に使う範囲**を決めて切り出します。この範囲を ROI（Region of Interest）と呼びます。

$$224 \le u' < 416, \qquad 214 \le v' < 406$$

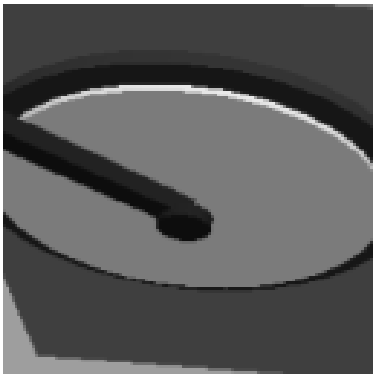
中心 $(320, 310)$、一辺 192 px の正方形です。上の不等号の右端は含まないので、横も縦も 192 画素です。



192×192 の ROI



グレースケール化



128×128 へ縮小

保存済み位置合わせ画像から、録画後に同じ前処理を行って作った説明図。表示上の各画像は同じ大きさに拡大している。新しい撮影ではない。

切り出した BGR 画像を、画素ごとの明るさだけを持つグレースケールへ変換します。その後、線形補間で 128×128 へ縮小します。固定寸法にすることで、毎回同じ数の特徴値を作れます。

分類前に、ほぼ空の画像を弾く

縮小後の明るさの**標準偏差 8 以上、かつ輪郭画素数 80 以上**の両方を確認します。輪郭画素数は Canny という輪郭検出処理で数えます。Canny の 2 閾値は 40 と 120 です。ほぼ単色、輪郭が少なすぎる画像を分類へ渡さないためで、正常・異常の区別ではありません。

保存例は標準偏差 40.72、輪郭画素 925 で、この確認を通過しました。条件不足なら判定不能として止め、正常というラベルには置き換えません。

次へ渡すもの　1 枚の 128×128 グレースケール画像を HOG へ渡し、分類に使える数値列を作ります。

HOG の入口：明るさの変化を測る

針や目盛りの形を分類器へ渡すには、画素をそのまま並べる以外に、輪郭の方向と強さをまとめる方法があります。HOG は Histogram of Oriented Gradients の略で、**画像の明るさの変化を方向別に数える特徴抽出処理**です。特徴量とは、分類に使いやすいよう画像から計算した数値です。HOG 自体が正常・異常を学習するモデルではありません。[2,4]

128×128 画像の位置 (u, v) の明るさを $I(u, v)$（0〜255）とします。内部の画素で、横と縦の**差分勾配値**を考えます。

$$g_x = I(u+1, v) - I(u-1, v), \quad g_y = I(u, v+1) - I(u, v-1)$$

これは注目画素を挟んだ左右・上下の画素の明るさの差です。OpenCV のこの計算は 2 画素間の差を **2 で割らない**ため、ここでの単位は明るさの値です。微分の近似を 1 px 当たりで定義するなら $2\,\mathrm{px}$ で割りますが、その式とは区別します。

計算例（説明用）：左 40、右 100、上 60、下 140 なら、$g_x = 60$、$g_y = 80$ です。変化の強さ m と方向 ϕ は

$$m = \sqrt{g_x^2 + g_y^2} = 100, \qquad \phi = \operatorname{atan2}(g_y, g_x) \simeq 0.9273\ \mathrm{rad} = 53.13^\circ$$

です。atan2 は、横と縦の成分から、象限も含めて方向角を求めます。その出力はそのまま 0〜π に限られるわけではありません。HOG の符号なし勾配では $\theta = \phi \bmod \pi$ として、180 度違う向きを同じ方向へまとめます。

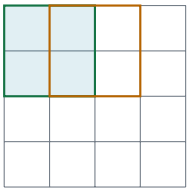
θ は**明るさが変わる方向**です。輪郭線に沿う方向とは通常 90 度違います。画像端の処理や角度近似を含む実計算は OpenCV 4.5.4 を使用し、上は内部画素の仕組みを示す例です。

> **次へ渡すもの**　各画素から、強さ m と 0〜180 度にまとめた方向 θ を得ます。次は小領域ごとに、方向別の強さへ集計します。

画素の方向を、小領域ごとの 9 個の数値へまとめる

1 画素の勾配だけでは、針や外枠の周囲の形を表せません。そこで 128×128 画像を、8×8 画素の **cell（セル）**という小領域として扱い、その近くの輪郭がどの方向に強いかをまとめます。

ヒストグラムは、値を区分に分けて足し合わせた表です。HOG では 0〜180 度を 9 区分に分け、各画素の強さ m を方向区分へ加えます。この加算を「投票」と呼びます。物体数の投票ではなく、**勾配強度を数値として足す**意味です。



1 マスは 8×8 画素の cell

緑：2×2cell の 1block

橙：8 px ずらした次の block

構造を示す模式図。実際の HOG 入力画像や学習結果の可視化ではない。

実装は近い 2 方向へ強さを分配します。区分の中心は 10、30、50、…、170 度です。例えば 53.13 度、強さ 100 なら、**方向分配だけを取り出した簡略例**では、50 度へ約 84.35、70 度へ約 15.65 を加えます。

実際には画素位置に応じて近い cell へも分配し、block 中心付近を重くする Gaussian 重みも使います。例の 100 をそのまま最終 HOG 値とは扱いません。

1cell 相当の方向表は 9 値です。2×2cell をまとめる **block（ブロック）**は $4 \times 9 = 36$ 値になります。block を重ねることで、同じ cell も周囲の異なる範囲と組にして数値化します。

次へ渡すもの　各 block の 36 値を、次の正規化へ渡します。明るさ・コントラストの違いの影響を抑えてから、全 block を並べます。

block を正規化し、1 画像を 8100 値へ変える

輪郭が同じでも、明るさの強弱でヒストグラム全体の値は変わります。**正規化**は、数値の大きさを一定の基準で割り、比較しやすくする処理です。HOG では block 内の 36 値を使い、局所的な強さをそろえます。入力 2 値に行った学習平均による標準化とは別です。

36 値を v_r、値番号を $r = 1, \ldots, 36$ とすると、OpenCV 4.5.4 の L2-Hys は、概略を丸めずに書けば次の段階です。[4]

$$v'_r = \frac{v_r}{\sqrt{\sum_{s=1}^{36} v_s^2 + 0.1 \times 36}}, \qquad c_r = \min(v'_r, 0.2)$$

$$\tilde{v}_r = \frac{c_r}{\sqrt{\sum_{s=1}^{36} c_s^2 + 0.001}}$$

まず 36 値の二乗和の平方根に安定化項を加えた値で割り、突出した値を 0.2 で抑え、もう一度割り直します。1 個だけ極端に強い輪郭に数値全体が左右されにくくします。

何個の数値を作るのか

入力 128×128、block16×16、移動幅（stride）8×8 なので、block の横位置は

$$N_u = \frac{128 - 16}{8} + 1 = 15, \qquad N_v = 15$$

です。最初の位置も数えるため 1 を足します。block は 225 個で、1 個 36 値なので

$$15 \times 15 \times (2 \times 2 \times 9) = 225 \times 36 = 8100$$

です。**8100 枚の画像や 8100 種類の物体ではなく、1 枚を表す 8100 個の数値**です。決まった順序に並べたベクトルを $\boldsymbol{h}(I) \in \mathbb{R}^{8100}$ と書きます。

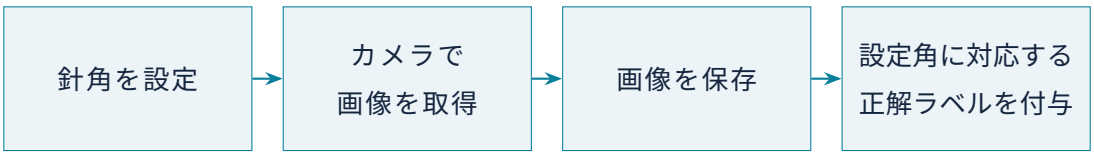
実設定は符号なし 9 方向、gamma 補正なし、L2-Hys 上限 0.2 です。特徴は float32 形式で、完成した 8100 値へ別の学習平均・標準偏差を適用する処理はありません。

> **次へ渡すもの**　1 枚の画像を表す 8100 値を SVM へ渡します。その SVM が何を学習したかを、教師画像の作り方から説明します。

分類の正解画像は、針角を設定して撮影する

正常・異常を分けるモデルには、操作の正解ではなく、**画像に対応するクラスの正解**が必要です。クラスは分類先の種類です。今回は正常（normal）と異常（abnormal）の 2 クラスです。

Gazebo 内の 3D ゲージは、箱・外枠・盤面・針の形状と配置を定義した仮想物体です。設定した針角にしてから、実行時と同じ形式のカメラで見た画像をシミュレータが描画し、ROS 画像として保存しました。画像生成 AI による写真合成ではありません。



クラス	収集に使った針角 [rad]
正常	$-0.13, -0.08, -0.03, 0.02, 0.07, 0.12$
異常	$0.72, 0.78, 0.84, 0.90, 0.96, 1.02$

この正常・異常は、**実験で設定した針位置のラベル規則**です。温度・圧力などの実際の物理量を測定し、一般の計器の安全基準と比較したものではありません。分類時に正解角や正解ラベルをモデルへ入力していません。

12 種類の「クラスと針角」の組について、固定した観測姿勢から、変更後 0.35 秒以降に、異なる画像時刻の 3 フレームを保存しました。2クラス $\times$ 6角度 $\times$ 3フレーム $=$ 36 画像ファイルです。

次へ渡すもの　保存画像とラベルを、学習用・検証用へ角度の組ごとに分けます。画像時刻が違っても、画像内容まで異なるとは限りません。

角度で分割し、学習画像だけを小さく加工する

同じ針角のほぼ同じ画像を学習用と確認用に分けると、見慣れた画像への成績を高く見積もる恐れがあります。そこで、**クラスと針角の組**を単位に分割しました。検証とは、学習に入れていないデータでモデルを確認することです。

分類用の分割	正常	異常
学習：各クラス 4 角度 ×3 画像	12 枚	12 枚
検証：各クラス別の 2 角度 ×3 画像	6 枚	6 枚

学習角度の番号は 0、1、3、4、検証は 2、5 で重複しません。ただし保存画像を照合すると、同じ針角の 3 枚は画素内容が同じでした。**学習 24 ファイルは 8 種類、検証 12 ファイルは 4 種類の画素内容**で、36 種類の独立した見た目を得たわけではありません。

学習用 24 枚にだけ、画像拡張を行う

画像拡張とは、小さな見た目の変化を加工で加え、学習へ含める方法です。元画像 1 枚をそのまま使い、さらに次の加工を組み合わせた 3 版を作ります。

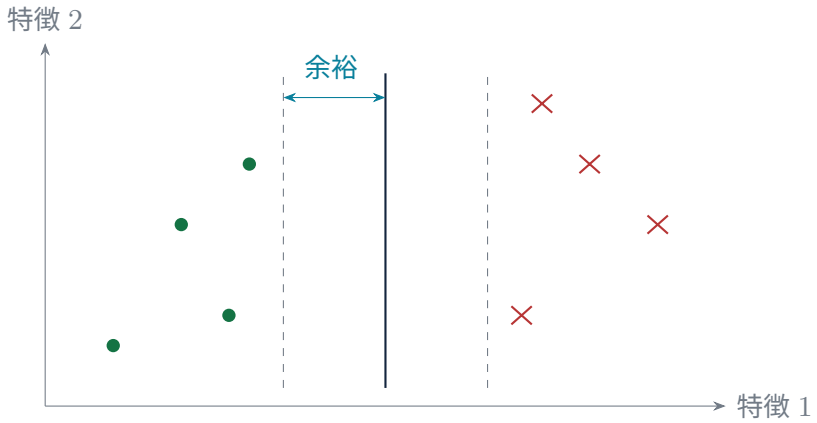
加工	各追加版の設定
回転・平行移動	回転 -2〜$+2$ 度、横・縦各 -3〜$+3$ px を乱数で選び、1 つの変換へ合成。
明るさ	$I_{\mathrm{new}} = \mathrm{clip}(\alpha I_{\mathrm{warp}} + \beta, 0, 255)$。$\alpha = 0.92$〜$1.08$、$\beta = -7$〜$+7$。
画像外・乱数	外側は反射して補い、整数画像へ変換。乱数 seed は 20261001。

24 枚 ×（元画像 1 ＋追加 3）＝ 96 学習例です。96 回の独立した撮影ではありません。検証 12 例は加工せず、拡張学習へ入れません。

次へ渡すもの　各学習画像へ前処理と HOG を適用し、96 組の「8100 値と正常・異常ラベル」を SVM 学習へ渡します。検証 12 例の個別結果記録は今回確認できず、単独の汎化精度は主張しません。

SVM は、2 クラスを分ける境界を学ぶ

HOG は輪郭を数値にしましたが、どの値なら正常かは決めていません。**SVM（Support Vector Machine）**は、特徴量と正解ラベルから、クラスを分ける境界を学ぶ機械学習モデルです。今回は**線形 SVM** で、特徴値を重み付きで足して判定します。[3,5]



中央：境界。破線：マージンの基準。

境界を理解するための 2 次元概念図。実測データや学習結果の可視化ではない。実際の特徴は 8100 次元。

境界のすぐ近くでだけ正解するより、正しい側に**余裕（マージン）**を持たせたいと考えます。ただし全例を厳密に分けられない場合もあるので、余裕不足や誤りをある程度許し、その量にも罰則を与えます。

学ぶものは 8100 個の重み $\boldsymbol{w}_{\mathrm{SVM}}$ と切片 b_{SVM} です。画像を保存してそのまま照合するだけの仕組みではありません。模倣回帰の重み $w_{\mathrm{BC},1}, w_{\mathrm{BC},2}$ とは別のモデル・別のパラメータです。

次へ渡すもの　次ページで「境界の余裕」と「許す不足」を式にします。ここからの教師は、補正操作ではなく正常・異常の正解ラベルです。

どんな境界がよいかを、式で決める

SVM の学習入力は、学習画像 i の HOG 列 $\boldsymbol{h}_i \in \mathbb{R}^{8100}$ です。数学上の正解符号を $t_i = +1$（正常）、-1（異常）とします。モデルの判定値は $f_i = \boldsymbol{w}_{\mathrm{SVM}}^{\top}\boldsymbol{h}_i + b_{\mathrm{SVM}}$ です。

$$\min_{\boldsymbol{w}_{\mathrm{SVM}},b_{\mathrm{SVM}},\boldsymbol{\xi}} \frac{1}{2}\|\boldsymbol{w}_{\mathrm{SVM}}\|_2^2 + C\sum_{i=1}^{96}\xi_i$$

$$t_i(\boldsymbol{w}_{\mathrm{SVM}}^{\top}\boldsymbol{h}_i + b_{\mathrm{SVM}}) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \qquad C = 1$$

これは使った C-SVC に対応するソフトマージンの目的関数です。$\boldsymbol{w}_{\mathrm{SVM}}$ は 8100×1、b_{SVM} と ξ_i は実数 1 個です。$\|\boldsymbol{w}_{\mathrm{SVM}}\|_2^2$ は 8100 個の重みの二乗和です。HOG は正規化済みの無次元の数値で、この判定関数と ξ_i も単位のない値です。

制約を日本語で読む

$t_i f_i$ は、正しい側なら正になる判定値です。ξ_i は**マージン制約の不足を許す量**です。必要最小の値は $\max(0, 1 - t_i f_i)$ になります。

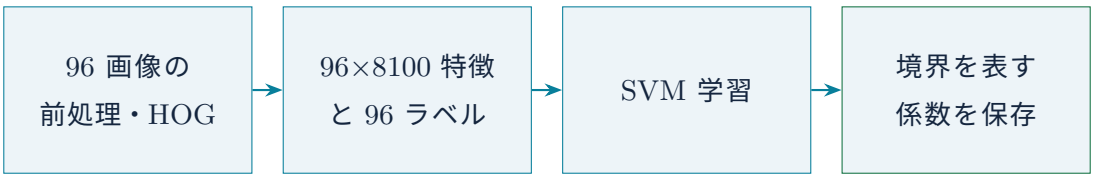
例（説明用）	意味
$t_i f_i = 1.4$	1 以上の余裕を満たす。必要な $\xi_i = 0$。
$t_i f_i = 0.4$	分類は正しいが余裕不足。必要な $\xi_i = 0.6$。
$t_i f_i = -0.2$	誤分類側。必要な $\xi_i = 1.2$。

$t_i f_i = 0$ は境界上です。$\xi_i > 0$ でも分類自体は正しい場合があります。第 1 項は重みを小さくして余裕の広い境界を求め、第 2 項は 96 例の不足を少なくします。C はその釣合いです。大きいほど不足を強く罰しますが、C だけで性能や過学習が決まるわけではありません。

次へ渡すもの　この目的と制約に従い、画像特徴とラベルから境界の係数を求めます。次ページで実際の学習設定を確認します。

実装では、96×8100 の数値表から境界を求めた

前ページの式を実行するため、1 枚分 8100 値を横 1 行にし、96 学習例を縦に並べました。この**96×8100 の特徴行列**と、各行に対応する正常・異常ラベルを OpenCV の学習 API へ渡しています。



実装設定	確認した値
ライブラリ	OpenCV 4.5.4
SVM の種類・カーネル	C_SVC、LINEAR（線形）
罰則係数	$C = 1.0$、クラス別の追加重みなし
学習 API	`svm.train(matrix, ROW_SAMPLE, labels)`
解法・上限	OpenCV の SMO 系の解法、最大 1000 反復
保存ラベル	normal=0、abnormal=1

SMO は、一部の変数ずつ更新して制約付き最適化を進める解法です。保存モデルには線形境界を表す係数が入ります。終了条件では最大反復数だけが有効な指定で、API へ渡した 10^{-6} は、有効な誤差終了フラグを伴っていません。実際の終了反復数は保存されていません。

カーネルは、入力同士を比較する計算の設定です。ここでは内積を使う線形設定で、画像や特徴をニューラルネットワークへ渡す構成ではありません。

分類器のメタデータには検証精度 1.0 という集計が残っていますが、画像別の確認結果を再照合できる記録が見つかりません。画像内容の重複もあるため、独立した汎化精度として提示しません。

次へ渡すもの　学習済み分類器を固定し、点検中に採用した別の画像の HOG 値へ適用します。学習画像や正解角を点検時に読み出して答えにする処理ではありません。

8100 個の積を足し、保存ラベルへ変換する

点検時の分類で受け取るのは、採用画像を前処理して得た $\boldsymbol{h}(I)$ です。I は分類する画像、$\boldsymbol{h}(I)$ はその 8100×1 の HOG 列です。保存した係数で次を計算します。

$$f(I)=\boldsymbol{w}_{\mathrm{SVM}}^{\top}\boldsymbol{h}(I)+b_{\mathrm{SVM}}=\sum_{r=1}^{8100}w_{\mathrm{SVM},r}h_r(I)+b_{\mathrm{SVM}}$$

r は特徴番号です。内積 $\boldsymbol{w}^{\top}\boldsymbol{h}$ は、同じ番号の重みと特徴値を掛け、8100 組の積を足す計算です。最後に切片を足した $f(I)$ が、境界のどちら側かを示す**符号付き判定値**です。

f そのものは確率でも、画素や実空間の距離でもありません。特徴空間での境界までの符号付き幾何距離なら $f/\|\boldsymbol{w}_{\mathrm{SVM}}\|_2$ です。実装はこの距離を撮影制御に使いません。

数学上の符号と、OpenCV の保存ラベルを区別する

本保存モデルのラベル順序と OpenCV の 2 クラス投票規則を確認すると、raw 判定値とクラスの対応は次です。

raw 判定値	API ラベル	画面へ表示する結果
$f>0$	0	normal：正常
$f\le0$	1	abnormal：異常

実運用は `predict` が返すクラスラベルを使い、raw 値の符号を独自のルールで推測していません。数学上の教師符号 $+1/-1$ と API の $0/1$ は、同じ番号ではありません。

動画の保存結果では、正常例は raw 値 $+0.998574$ でラベル 0、異常例は -1.022747 でラベル 1 でした。これは保存結果の読取りで、新しい分類推論ではありません。絶対値を「正常確率」「異常度」とは読みません。設定の最小 raw 絶対値は 0 で、確率による棄却閾値ではありません。

次へ渡すもの　分類ラベルを記録し、画面表示へ渡します。分類完了の正確な時刻は残っていないため、動画では「この試行の保存結果」として示します。

1 つの保存例で、情報の受け渡しを追う

ここまでの部品を、動画の異常例 1 試行へ戻してつなぎます。箱は固定し、模倣モデルの係数を変更せず 2 回補正しました。下表は、同じ試行の観測と操作です。

観測	中心 u [px]	e_x [px]	次の補正 [rad]
初期観測後	242	−78	+0.0464982
1 回目補正後	274	−46	+0.0313344
2 回目補正後	295	−25	なし
続く安定画像	295	−25	なし

観測→位置補正

元画像から固定画像処理が外枠を検出し、目標 320 px との差を計算します。左右ずれと実測関節 1 角を学習済み模倣モデルへ入れ、差分を予測します。安全検査と MoveIt の計画を通った軌道を controller が実行し、新しい画像へ進みます。

再観測→撮影画像の採用

2 回後のずれは $(-25, -3.5)$ px、幅 234 px でした。続く新しい画像も条件を満たして安定していたため、その元画像を採用しました。受信するだけの画像、補正に使う画像、採用する画像は、このように役割が違います。

採用画像→分類→復帰

採用画像の枠を位置合わせし、ROI 切出し・白黒化・縮小後、HOG が 8100 値を作ります。学習済み線形 SVM が異常ラベルを返しました。保存した画像と結果の対応を確認し、固定の home 復帰を完了しました。

初期移動 1 回＋模倣補正 2 回＋ home 復帰 1 回で、全軌道送信は 4 回です。初期移動と復帰を模倣モデルの補正へ数えていません。

次へ渡すもの　この一連の処理は、2 つの学習モデルと複数の固定処理をつないだ自動点検です。次ページで担当を整理し、その後で比較実験を読みます。

何を学習し、何を固定の処理が担当するか

部品が「何を受け取り、何を次へ渡すか」を整理します。AI がすべてを一括で動かすという構成ではなく、学習済みの 2 モデルを固定処理と接続しています。

担当	受け取るもの	計算して次へ渡すもの
固定の画像処理	手先カメラ画像	外枠候補を条件で絞り、中心・幅・ずれを計算。
学習済み模倣モデル	左右ずれと現在の関節 1 角	次に回す関節 1 の角度差分。
安全検査と MoveIt	差分と実測状態、周囲の形状	値・関節・衝突条件を確認した軌道。
ROS controller	関節角と時刻の列	軌道に沿う動作。実測状態を使って到達を確認。
撮影品質確認	到達後の新しい検出付き画像	2 枚の安定条件を満たした採用元画像。
固定の前処理・HOG	採用画像	位置合わせ・切出し等を行い、8100 特徴値。
学習済み線形 SVM	8100 特徴値	保存された正常・異常クラス。

2 つの学習は、正解の種類が違う

模倣モデルは**状態→操作**の対応を学び、学習する数値は 3 個の回帰係数です。分類モデルは**画像特徴→正常・異常**の対応を学び、8100 個の重みと切片で境界を表します。

どちらも、用意した正解を使う教師あり学習です。報酬を得て方策を更新する強化学習は、今回の完成機能には含めません。対象は既知領域内の単一ゲージで、未知空間の自由探索や画像による個体識別を示すものではありません。

次へ渡すもの　次は、固定姿勢・規則制御・模倣制御を同じ評価条件で比べ、撮影位置補正が何を改善したかを確認します。

比較実験で、何を確かめたいのか

実験の問いは 2 つです。**固定姿勢で撮るだけの場合に対し、画像を見て補正すると撮影条件を満たせるか。そして、教師の規則方式と、教師操作を学んだ模倣方式で、結果がどう違うか。**

比べる方式	初期観測後の操作の決め方
固定観測	位置合わせ補正をしない。撮影品質不足なら分類しない。
規則制御	その場で比例式 $-0.5e_x/J$ を計算し、上限を通した補正を選ぶ。
模倣制御	学習済みリッジ回帰へ左右ずれと現在角を入れ、上限を通した補正を選ぶ。

「規則制御」は、人が決めた計算ルールに従う制御です。本比較では教師にも使った比例式を指します。模倣方式から規則方式へ自動で切り替える処理はありません。

共通にした条件

最終評価の左右 2 配置を、それぞれ 3 方式で使いました。方式ごとに初期状態へ準備し、同じ初期観測姿勢、カメラ、針条件、検出・撮影品質条件、分類器を用いました。対象は各試行の準備で配置し、試行中は固定します。

補正する方式は関節 1 のみ、要求 cap± 0.0548 rad、安全上限 ± 0.055 rad です。1 試行の補正は最大 4 回、全軌道送信は 8 回または 300 秒が上限で、home 用の予算を残します。初期姿勢への移動と home 復帰も全送信数に含みます。

固定観測に補正を追加したり、失敗後に撮影品質の条件を緩めたりはしていません。画像条件不足は比較結果として保持しています。

> **次へ渡すもの**　何の成績を見るかを次ページで定義します。評価とは、設定を固定した後の新しい条件で、どこまでできるかを測ることです。

学習・調整・最終評価を分け、数え方を決める

学習した配置をそのまま成績確認に使うと、既に見た状態への当てはまりを、未知の状態への性能と思い違いしやすくなります。そのため、配置の用途を分けました。

用途	データ	何を決めるか
学習	左右 × 小中大の 6 配置。11 完了試行・17 教師操作。	予測係数、平均、標準偏差。
調整	別の左右 2 配置。2 試行・4 教師操作。	λ の候補を選択。
最終評価	さらに別の左右 2 配置 ×3 方式＝ 6 試行。	設定を凍結した後の成績。

最終評価のデータは学習・調整へ入れません。評価結果を見て配置やモデルを変えて再評価した成績ではありません。動画の再現確認を、新しい未学習評価へ加算もしません。

結果表の列を、先に定義する

- **撮影成功**：撮影品質と 2 画像の安定条件を満たし、検査用画像を採用できた試行。成功数を方式の全試行数で割る。
- **分類正答**：実際に分類した試行のうち、実験の針クラスと結果が一致。未分類はこの分母から除く。
- **補正回数**：位置合わせの軌道を実際に送った回数。**全送信数**：固定の初期移動・補正・復帰すべての軌道送信数。
- **所要秒**：1 回の点検全体の経過時間。**home 復帰**：終了姿勢への到達確認結果。

分類できた画像だけの正答率が高くても、多くが未撮影なら点検全体の成功とは言えません。そのため、**画像採用・正しい分類・home 復帰をすべて満たす点検完了数**も併記します。

> **次へ渡すもの**　以上の定義で、全 6 試行の保存結果を読みます。故障停止や撮影品質不足を削除して、成功率の分母を小さくはしません。

左右 2 配置、全 6 試行で得た結果

表の「撮影」は前ページの撮影品質・安定条件に対する成否です。「異常・正答」は、実際の分類結果が異常で、設定した針クラスと一致したことを表します。「未実施」は正常という意味ではありません。

1 行を文章で読む

左配置の模倣方式は、モデルを 2 回呼び、2 回の補正で撮影条件を満たしました。採用画像を異常と正しく分類し、home へ戻りました。全送信は、固定の初期移動 1 ＋補正 2 ＋復帰 1 ＝ 4 回で、全体は 95.7 秒でした。この読み方で全 6 行を比べます。

配置	方式	撮影	分類	補正	全送信	秒
左	固定	不足	未実施	0	2	40.7
左	規則	成功	異常・正答	2	4	95.1
左	模倣	成功	異常・正答	2	4	95.7
右	固定	不足	未実施	0	2	46.6
右	規則	成功	異常・正答	2	4	92.2
右	模倣	成功	異常・正答	2	4	94.2

全 6 試行で home 復帰は成功。秒は保存値を小数 1 桁に丸めた。

次へ渡すもの　各試行の結果を、そのまま残して方式ごとに集計します。次ページでは撮影・分類・点検完了の分母をそろえて読みます。

分類できた画像だけで、全体の成功を判断しない

比較したいのは、画像分類器が何回正解したかだけでなく、撮影画像を確保して点検を終えられたかです。前ページの 6 試行を、方式ごとの全 2 試行を基準に読み直します。

指標	固定	規則	模倣
撮影成功／全試行	0/2	2/2	2/2
分類正答／分類実施	0/0（未分類）	2/2	2/2
点検完了／全試行	0/2	2/2	2/2
補正送信合計	0	4	4
模倣モデル呼出し合計	0	0	4

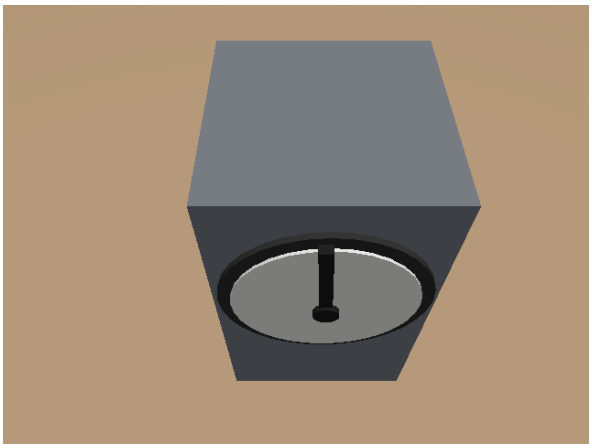
固定姿勢だけでは両配置で撮影条件に届かず、画像を見て補正した規則・模倣では両配置で採用・分類できました。**規則と模倣の成功数・補正回数は同じ**で、時間も近い値です。2 配置という少数例から、模倣の優越性や広い配置での成功率は断定できません。固定の分類正答は 0/0 で、割合は計算できません。分類しなかった 2 件を「間違えなかったから正常」と扱わず、点検完了では 0/2 として数えます。規則と模倣は、画像採用・正しい分類・home 復帰を全 2 件で満たした結果です。

全送信数の合計は固定 4 回、規則 8 回、模倣 8 回でした。模倣のモデル呼出しは合計 4 回ですが、規則の補正はモデル呼出しをせず、その場の比例式で選んでいます。補正回数とモデル呼出し回数が、いつも同じ意味とは限りません。

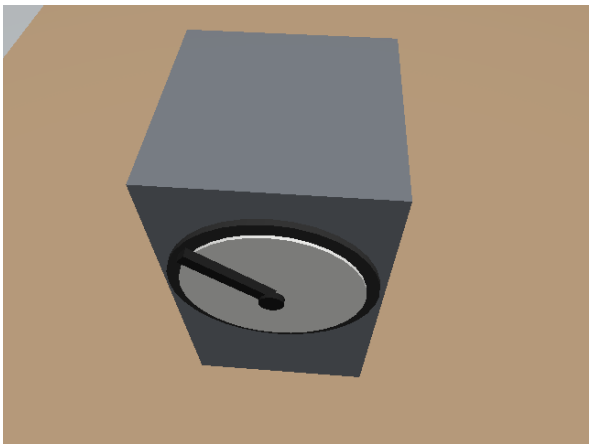
次へ渡すもの　これは同じ評価配置で方式を比べた成績です。次の動画の正常・異常 2 事例は、役割が違う紹介用記録として読みます。

動画の正常例と異常例が示していること

動画は、正常・異常それぞれの採用画像と実際の分類結果を示します。**別の点検試行**であり、箱の針が同一試行の途中で変わった映像ではありません。



正常例の採用元画像



異常例の採用元画像

正常例：撮影した画像から正常の結果が出た

初回観測から撮影品質を満たし、新しい 2 画像の安定を確認して採用しました。保存された分類結果は正常です。**模倣モデルの呼出し 0 回・補正 0 回**なので、模倣モデルが補正して正常を得た実証には数えません。

異常例：撮影位置を補正してから異常を分類した

初回の水平ずれは -78 px でした。模倣モデルの呼出しと補正を 2 回行い、-25 px まで近づいた安定画像を採用しました。保存結果は異常で、固定の home 復帰も完了しました。既知配置での再現・記録確認の試行です。

2 事例は撮影配置や制御モデル設定などが同一ではありません。**正常・異常の違いだけで動作の差が生じたとは判断できません。**正常例では補正方策が呼ばれていない事実も含め、同一モデルの正常・異常対照実験にはしません。

画像分類器の版とファイル内容は両事例で同一と照合できました。分類完了の正確な時刻は不明なので、動画の結果欄は「この試行の保存結果」として示します。これらを前ページの最終評価 6 試行には加算しません。

次へ渡すもの　動画は処理がどうつながるかを示す実例です。制御方式の成績比較には、同じ評価配置で比べた前ページの実験を使います。

実験の問いへ戻って、分かった範囲を読む

問い 1：画像を見て補正すると撮影できるか

最終評価の左右 2 配置では、固定観測は撮影 0/2、規則・模倣は各 2/2 でした。対象固定のまま手先視点を変え、画像条件を満たしてから分類する一連の処理を、シミュレーション内で確認しました。撮影条件不足を正常扱いせず、撮影品質とゲージ状態の分類を分けたことが工夫です。

問い 2：教師の規則方式より模倣方式がよいか

規則方式は、人が決めた比例式に従います。模倣方式は、11 完了試行・17 教師操作から、その状態→操作対応を回帰式で近似しました。左右で符号が変わり、小さい補正量と大きい補正量を含むデータを使いました。ただし最終 2 配置では成功数も補正回数も同じで、**規則制御への優越性は未確認**です。

何を広げるときに、追加の証拠が必要か

学習したのは関節 1 の水平補正だけです。上下ずれ・大きさの調整、未検出状態からの自由探索、広い配置・照明条件、実機への一般化は確認していません。初期移動と home 復帰は固定動作であり、腕のすべてを学習で制御する作品ではありません。

分類画像には同じ画素内容の重複があり、独立した分類精度の画像別報告も再確認できません。正常・異常はこの実験の針位置ラベルで、一般の計器の危険度や安全基準を判定する機能ではありません。

安全面では、モデル出力の上限だけでなく、途中の計画点・衝突・到達を確認し、異常時に送信を止める構成にしました。検査した条件内での確認であり、すべての現実環境の安全を保証する表現はしません。

> **次へ渡すもの**　作品として確認したのは、「固定対象を撮りやすい視点へ補正し、採用画像を別の学習モデルで分類する」一連の処理です。発展には、用途に応じた独立データと安全検証が必要です。

参考にした公開資料と、実装の入口

本文は保存した実装・設定・モデル・実記録へ照合しています。以下は基礎概念とソフトウェアの一次資料です。本文の計算や数値例は、これらを別に読まなくても追えるよう記載しました。

[1] T. Osa ほか, *An Algorithmic Perspective on Imitation Learning*, 2018. 模倣学習と behavior cloning の概念。
https://arxiv.org/abs/1811.06711

[2] N. Dalal, B. Triggs, *Histograms of Oriented Gradients for Human Detection*, CVPR 2005. HOG の基本設計。
https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf

[3] C. Cortes, V. Vapnik, *Support-vector networks*, Machine Learning 20, 1995. マージンと制約付き分類学習。
https://doi.org/10.1007/BF00994018

[4] OpenCV 4.5.4, HOG 実装。方向・位置分配、正規化の確認。
https://github.com/opencv/opencv/blob/4.5.4/modules/objdetect/src/hog.cpp

[5] OpenCV 4.5.4, SVM 実装。C-SVC、学習解法、2 クラス投票・raw 値の確認。
https://github.com/opencv/opencv/blob/4.5.4/modules/ml/src/svm.cpp

[6] Gazebo Fortress, Sensors. シミュレータとセンサ。
https://gazebosim.org/docs/fortress/sensors/

[7] MoveIt 2 Humble, Motion Planning Pipeline. 動作計画の構成。
https://moveit.picknik.ai/humble/doc/examples/motion_planning_pipeline/motion_planning_pipeline_tutorial.html

提出予定の公開コード内の相対パス

検出と画像位置合わせ：`active_inspection/detection.py`。模倣モデル：`active_inspection/range_extension/policy.py`。分類前処理と推論：`multigauge/vision.py`。非公開資料や PC 上のフォルダーを参照しなくても、目的から結果まで本文内で理解できる構成です。